

课程笔记

CS294-277 课程笔记: Lecture 11B 灵巧操作案例 (下)

Codex

2026-04-16

Learning Manipulation

Jitendra Malik

视频作者/频道: CS294-277 / UC Berkeley

发布日期: 2024-11-25

视频时长: 未在本地元数据中提供

视频链接: <https://www.youtube.com/watch?v=mkI7WGgj5xA>

目录

1 本讲定位与核心矛盾	2
1.1 为什么长时序操控比单步操控更难	2
1.2 本讲的中心假设	2
1.3 本章小结	3
2 ManipGen 的局部性假设	3
2.1 从全局任务到局部交互	3
2.2 局部策略的定义	3
2.3 局部策略的三个不变性	3
2.4 本章小结	3
3 从 RL 学局部技能	3
3.1 局部 skill library	3
3.2 奖励设计与 PPO	4
3.3 为什么这一步适合用 RL	4
3.4 本章小结	4
4 从专家到通用策略的蒸馏	4
4.1 多任务 DAgger	4
4.2 观测设计：为什么只用局部信息	4
4.3 网络结构	5
4.4 本章小结	5
5 部署：VLM、运动规划与局部策略	5
5.1 Text-conditioned manipulation	5
5.2 为什么语言在这里有用	5
5.3 Zero-shot performance	5
5.4 本章小结	5
6 类似案例：AutoMate	6
6.1 为什么它和 ManipGen 是同一类思路	6
6.2 几个值得记住的想法	6
6.3 本章小结	6
7 总结与延伸	6
7.1 本章小结	6

Learning Manipulation

Jitendra Malik

图 1: 课程官方材料中的代表性页面，用于帮助读者在进入本讲正文前建立整体上下文。

1 本讲定位与核心矛盾

这节课延续 11A 的灵巧操作主题，但关注点从“监督学习怎么做”切换到了“当任务拉长到多阶段、多对象、多场景时，系统如何仍然能稳定工作”。讲者选取的主案例是 Dalal et al. (2024) 的 ManipGen，它试图让一个单一 generalist agent 在真实场景中完成 zero-shot long-horizon manipulation。

1.1 为什么长时序操控比单步操控更难

长时序任务的难点不只是动作序列更长，而是任务会跨越多个层次：

- 先要理解任务目标；
- 再要决定子任务顺序；
- 然后要在局部接触中完成稳定控制；
- 最后还要在失败时恢复并继续前进。

课程摘录指出，若想靠纯数据规模解决这类问题，数据集往往需要增长到 100K–1M trajectories 的量级，这对人工采集几乎不可持续。另一方面，sim2real (simulation-to-reality) 也不是自然成立的，因为接触动力学、场景组合和长时序行为都很难精确模拟。

1.2 本讲的中心假设

ManipGen 的关键判断

许多操作任务可以分解为“接近目标”和“在目标附近发生接触”两个阶段。只要把第二阶段限制在局部坐标系和局部观测里，许多看似复杂的问题就会突然变得更可学。

1.3 本章小结

本讲的主线不是“更多端到端学习”，而是“更合理的分层与局部化”。ManipGen 的整个系统设计都围绕这个原则展开。

2 ManipGen 的局部性假设

2.1 从全局任务到局部交互

很多家庭和工业操作任务都可以拆成两步。比如开门：

1. 先把手移动到门把附近的预接触位姿；
2. 再在门把附近执行抓握、旋转、拉动等接触丰富的动作。

对第二步来说，机器人并不需要知道整个房间的几何细节，只要知道门把和手的相对关系、附近局部视觉信息以及动作限制即可。这就是 locality of interaction（交互局部性）。

2.2 局部策略的定义

讲义给出的核心形式是局部策略（local policy）：

$$\pi_{\text{loc}}(a_{\text{loc}}^t \mid O_{\text{loc}}^t)$$

它只看局部观测 O_{loc}^t ，并输出局部动作 a_{loc}^t 。如果初始化时末端执行器已经足够接近目标，那么局部策略就可以忽略绝对位姿，把注意力放在相对几何和接触上。

2.3 局部策略的三个不变性

课程摘录强调，局部策略对三类变化更鲁棒：

- **Absolute pose**：只关心相对位置，不依赖全局位姿；
- **Skill order**：任务链可以由外部 motion planning 或高层策略组织；
- **Scene configuration**：只观察目标附近的局部区域，不必建模整场景。

2.4 本章小结

ManipGen 的重要贡献，不是“把所有事情都学进一个大模型”，而是把长时序操控拆成可重复调用的局部模块。

3 从 RL 学局部技能

3.1 局部 skill library

ManipGen 的第一步是先通过强化学习（RL）在模拟中学到一组局部技能库。讲义列出的五个技能是：

- pick

- place
- grasp handle
- open
- close

这些技能之所以可以单独学习，是因为每个技能都能被局部状态、局部目标和局部奖励描述。

3.2 奖励设计与 PPO

课程摘录给出一个复合奖励形式：

$$r = c_1 r_{ee} + c_2 r_{obj} + c_3 r_{ee,obj} + c_4 r_{action} + c_5 r_{succ}$$

其中 r_{ee} 、 r_{obj} 和 r_{succ} 分别强调末端执行器、目标对象以及成功性； $r_{ee,obj}$ 和 r_{action} 则用于约束运动与便于 sim2real。

这些局部策略使用 privileged observation 和 proprioceptive state 训练，并采用 PPO (Proximal Policy Optimization, PPO) 作为优化器。这样做的好处是，RL 在简单局部技能上可以提供密集反馈，而且不需要大量人工演示。

3.3 为什么这一步适合用 RL

局部技能的关键不在于全局任务多么复杂，而在于局部接触是否可以被稳定学习。只要局部状态足够简洁，RL 的探索和优化就比从人类示范里直接学一个超长行为序列更自然。

3.4 本章小结

ManipGen 先把“难的长任务”拆成“容易学的局部技能”，再用 RL 把这些技能做扎实。这是整个方法能成立的第一层基础。

4 从专家到通用策略的蒸馏

4.1 多任务 DAgger

在学到局部专家后，ManipGen 进一步用在线模仿学习 (online imitation learning) 和多任务 DAgger (Dataset Aggregation, DAgger) 把这些单任务局部专家蒸馏成更一般的 visuomotor policy。

这里的逻辑很重要：不是直接拿一个通用策略从头学，而是先让局部专家在各自的窄域内做到可靠，再把它们的行为知识迁移到更一般的策略中。

4.2 观测设计：为什么只用局部信息

蒸馏阶段的观测设计并不是“越多越好”，而是“与局部目标最相关的信息越够用越好”。课程摘录提到，ManipGen 主要使用：

- local wrist camera information;
- current frame image;

- first frame depth image;
- first frame semantic image。

其中 Grounded SAM (Grounded Segment Anything Model) 被用来提取语义信息, 从而帮助定位目标对象。这样一来, 系统并不需要依赖过重的端到端视觉编码, 而是可以把一部分视觉理解交给成熟的计算机视觉模块。

4.3 网络结构

蒸馏后端使用的是 ResNet18 + MLP 的组合。这个选择本身也说明了一种工程原则: 当高层任务已经通过结构化分解被简化时, 末端策略并不一定非要使用最复杂的网络。

4.4 本章小结

蒸馏阶段的本质, 是把“局部专家能做”的知识转成“通用策略会做”的参数化表达, 同时保留局部观测和任务分解的结构优势。

5 部署: VLM、运动规划与局部策略

5.1 Text-conditioned manipulation

ManipGen 的最终部署不是一个单模型直接端到端出动作, 而是三段式的 hierarchical decomposition:

1. 用 VLM (vision-language model) 根据场景和文本提示做高层规划;
2. 用 neural motion planning 把机器人带到局部可操作位姿;
3. 用 local policies 完成局部接触动作;

然后把这三段循环起来, 直到完成整段长时序任务。

5.2 为什么语言在这里有用

讲义明确指出, language model 可能并不擅长直接输出低层动作, 但它在高层规划上很有用。原因是语言更适合表达目标、子步骤和操作约束, 而不是毫秒级的控制细节。

5.3 Zero-shot performance

课程摘录给出的结果显示, ManipGen 在多个真实任务上取得了 76% 的 zero-shot success rate, 平均完成 4.28 / 4.8 个阶段。相比之下, OpenVLA、SayCan、LLMTrajGen 和 VoxPoser 在这些任务上的表现更弱。

这类结果真正说明的是: 当任务足够长、足够组合化时, 层级式分解和局部技能库往往比单一模型更可靠。

5.4 本章小结

部署阶段证明了一个现实判断: 高层规划、运动规划和局部控制各做各的, 反而比试图让一个模型一次性处理所有层级更稳。

6 类似案例：AutoMate

6.1 为什么它和 ManipGen 是同一类思路

讲义最后提到 AutoMate2，说明类似的“先学局部专家，再蒸馏和组合”的策略也能用于 assembly 和 insertion 一类任务。这里的关键仍然是：RL 更适合窄空间、清晰 privileged information 的局部问题，而 imitation learning 更适合把这些局部能力推广成更一般的策略。

6.2 几个值得记住的想法

课程摘录提到三点很有意思：

- dis-assembly 可以反向用来得到 assembly actions；
- human demonstrations 可以作为 RL 的初始化；
- dynamic time wrapping（动态时间规整）可以帮助对齐人类与机器人轨迹。

这些想法都在说明一个共同点：当任务复杂度上升时，最有效的办法通常不是直接暴力拟合，而是把问题拆成可对齐、可逆、可蒸馏的子问题。

6.3 本章小结

AutoMate 不是在讲一个完全不同的故事，而是在用另一个任务家族证明同一个观点：层级式、模块化的学习路线更适合真实机器人长时序操作。

7 总结与延伸

这一讲把灵巧操作的讨论从“单步学习”推进到了“长时序系统设计”。ManipGen 的核心启发可以概括为三句话：

- 先把任务拆成局部接触和高层规划；
- 先用 RL 学局部专家，再用 DAgger 蒸馏到通用策略；
- 部署时再把 VLM、运动规划和局部策略串起来。

如果把 11A 和 11B 合起来看，这两讲其实共同建立了一个更成熟的机器人学习观：真正可扩展的系统，很少是纯端到端的，而更像是“局部技能 + 高层组织 + 适当感知”的组合体。

7.1 本章小结

长时序操控的关键，不是让一个大模型替代所有模块，而是让不同模块各自承担它最擅长的层次。